



Unit-2 - Image Enhancement: Spatial domain methods: Point processing, Intensity transformations.

Digital Image Processing (Teerthanker Mahaveer University)



Scan to open on Studocu

Image enhancement refers to the process of highlighting certain information of an image, as well as weakening or removing any unnecessary information according to specific needs. For example, eliminating noise, revealing blurred details, and adjusting levels to highlight features of an image.

Image enhancement techniques can be divided into two broad categories:

- **Spatial domain** — enhancement of the image space that divides an image into uniform pixels according to the spatial coordinates with a particular resolution. The spatial domain methods perform operations on pixels directly.
- **Frequency domain** — enhancement obtained by applying the Fourier Transform to the spatial domain. In the frequency domain, pixels are operated in groups as well as indirectly.



This chapter discusses the image enhancement techniques implemented in the **spatial domain**.

Types of Spatial Domain Technique

Types of spatial domain operator:

Point operation (intensity transformation) - Point operations refer to running the same conversion operation for each pixel in a grayscale image. The transformation is based on the original pixel and is independent of its location or neighboring pixels.

Point Operation

Point operations are often used to change the grayscale range and distribution. The concept of point operation is to map every pixel onto a new image with a predefined transformation function.

$$g(x, y) = T(f(x, y))$$

- $g(x, y)$ is the output image
- T is an operator of intensity transformation
- $f(x, y)$ is the input image

Basic Intensity Transformation Functions

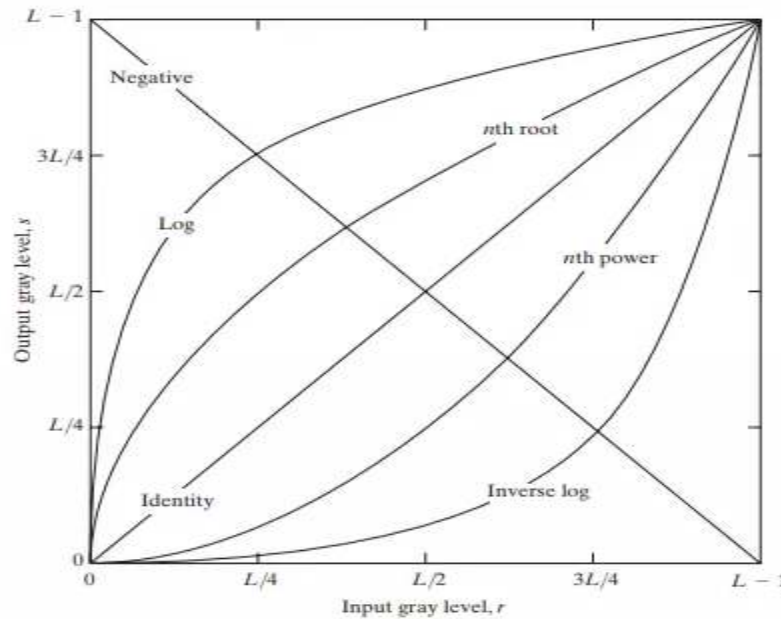
The simplest image enhancement method is to use a 1×1 neighborhood size. It is a point operation. In this case, the output pixel ('s') only depends on the input pixel ('r'), and the point operation function can be simplified as follows:

$$s = T(r)$$

Where T is the point operator of a certain gray-level mapping relationship between the original image and the output image.

- s, r : denote the gray level of the input pixel and the output pixel.

Different transformation functions work for different scenarios.



Linear

Linear transformations include identity transformation and negative transformation.

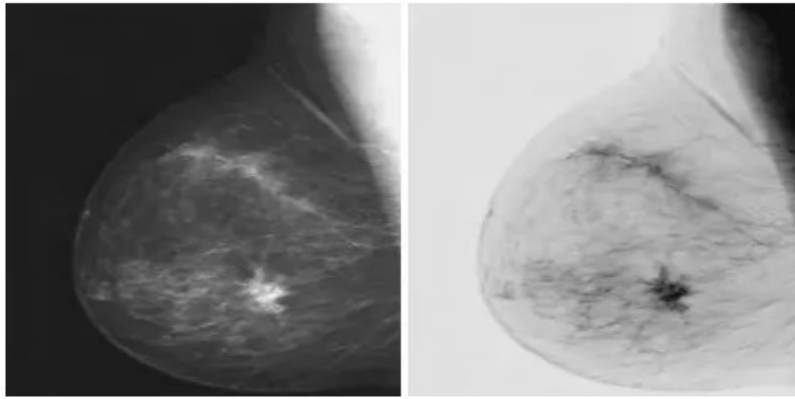
In identity transformation, the input image is the same as the output image.

$$s = r$$

The negative transformation is:

$$s = L - 1 - r = 255 - 1 - r = 254 - r$$

L = Largest gray level in an image. The negative transformation is suited for enhancing white or gray detail embedded in dark areas of an image, for example, analyzing the breast tissue in a digital mammogram.



1. Logarithmic

a) General Log Transform

The equation of general log transformation is:

$$s = c * \log(1 + r)$$

Note:

- s,r: denote the gray level of the input pixel and the output pixel.
- 'c' is a constant; to map from [0,255] to [0,255], $c = 255/\text{LOG}(256)$
- the base of a common logarithm is 10

In the log transformation, the low-intensity values are mapped into higher intensity values. It maps a narrow range of low gray levels to a much wider range. Generally speaking, the log transformation works the best for dark images.

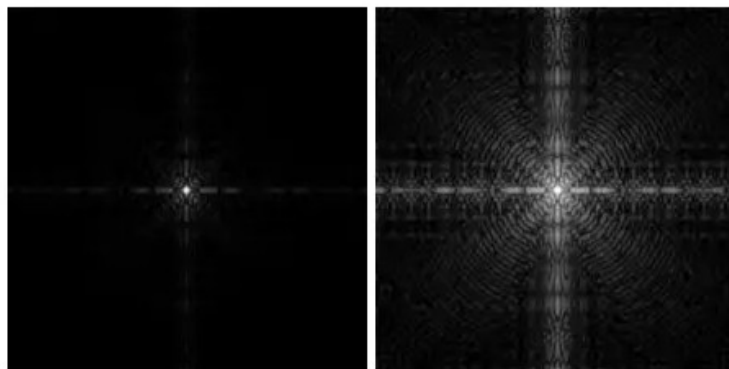


Image source: [Slideshare.net](https://www.slideshare.net)

b) Inverse Log Transform

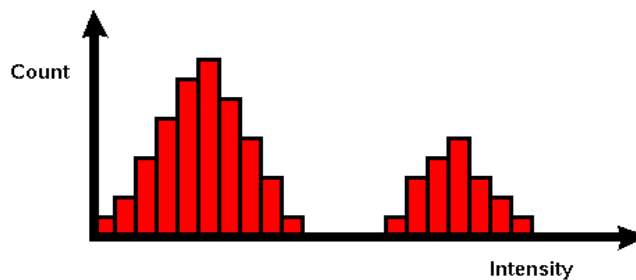
The inverse log transform is opposite to log transform. It maps a narrow range of high gray levels to a much wider range. The inverse log transform expands the values of light-level pixels while compressing the darker-level values.

$$s = \text{power}(10, r * c) - 1$$

Note:

- s, r : denote the gray level of the input pixel and the output pixel.
- 'c' is a constant; to map from $[0, 255]$ to $[0, 255]$, $c = \text{LOG}(256)/255$

Histogram



Common Names: Histogram

In an image processing context, the histogram of an image normally refers to a histogram of the [pixel intensity values](#). This histogram is a graph showing the number of [pixels](#) in an image at each different intensity value found in that image.

For an 8-bit [grayscale image](#) there are 256 different possible intensities, and so the histogram will graphically display 256 numbers showing the distribution of pixels amongst those grayscale values. Histograms can also be taken of [color images](#) --- either individual histograms of red, green and blue channels can be taken, or a 3-D histogram can be produced, with the three axes representing the [red, blue and green](#) channels, and brightness at each point representing the pixel count. The exact output from the operation depends upon the implementation --- it may simply be a picture of the required histogram in a suitable image format, or it may be a data file of some sort representing the histogram statistics.

Note:

A histogram is a graph. A graph that shows frequency of anything. Usually histogram have bars that represent frequency of occurring of data in the whole data set.

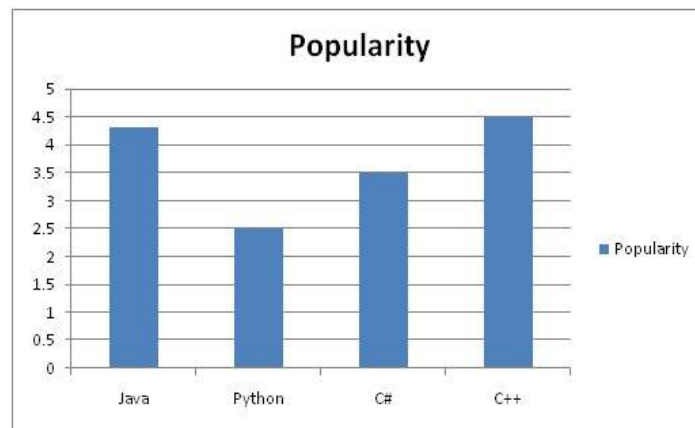
A Histogram has two axis the x axis and the y axis.

The x axis contains event whose frequency you have to count.

The y axis contains frequency.

The different heights of bar shows different frequency of occurrence of data.

Usually a histogram looks like this.



Histogram Processing Techniques

Histogram Sliding

In Histogram sliding, the complete histogram is shifted towards rightwards or leftwards. When a histogram is shifted towards the right or left, clear changes are seen in the brightness of the image. The brightness of the image is defined by the intensity of light which is emitted by a particular light source.

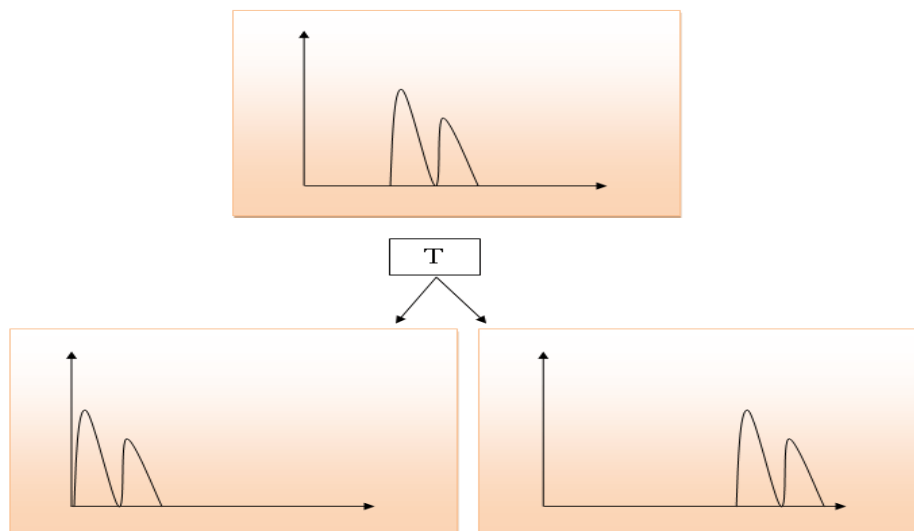


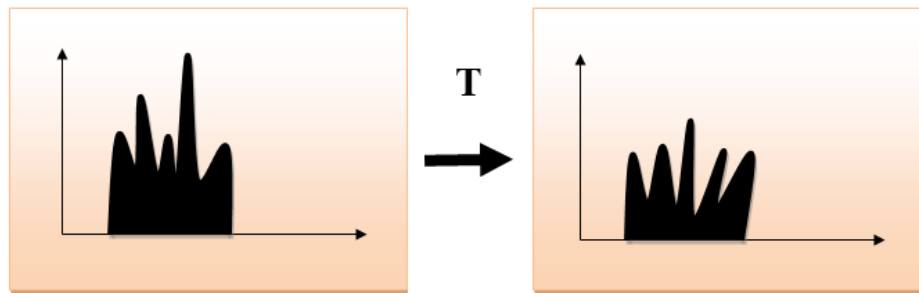
Fig. Histogram Sliding

Histogram Stretching

In histogram stretching, contrast of an image is increased. The contrast of an image is defined between the maximum and minimum value of pixel intensity.

If we want to increase the contrast of an image, histogram of that image will be fully stretched and covered the dynamic range of the histogram.

From histogram of an image, we can check that the image has low or high contrast.

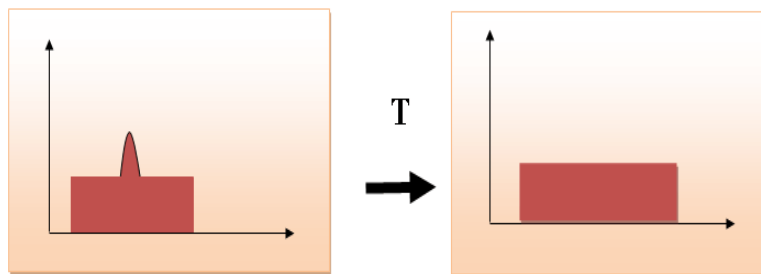


Histogram Equalization

Histogram equalization is used for equalizing all the pixel values of an image. Transformation is done in such a way that uniform flattened histogram is produced.

Histogram equalization increases the dynamic range of pixel values and makes an equal count of pixels at each level which produces a flat histogram with high contrast image.

While stretching histogram, the shape of histogram remains the same whereas in Histogram equalization, the shape of histogram changes and it generates only one image.



Histogram equalization is used to enhance contrast. It is not necessary that contrast will always be increase in this. There may be some cases were histogram equalization can be worse. In that cases the contrast is decreased.

Image Averaging for Noise Reduction

One of the more interesting applications of the image averaging (or image addition) operation is suppressing the noise component of images.

In this case, the addition operation is used to take the average of several noisy images that are obtained from a given input image. Assume that our desired input image is $I[x, y]$. The imaging process and the quantization operation of the utilized A/D converter lead to a noisy image $I_{\text{noisy}}[x, y]$.

Assume that the noise effect can be modeled as a noise image, $n[x, y]$, that is added to the noise-free input $I[x, y]$. This gives us

$$I_{\text{noisy}}[x, y] = I[x, y] + n[x, y]$$

If we capture the same input image several times, the noise-free image $I[x, y]$ will remain the same while the noise component will vary from one capture to the other. If we assume that the noise components are not correlated with each other and the mean value of the noise is zero, then averaging several noisy

images should suppress the noise. This is due to the fact that $I[x, y]$ is the same from one capture to the other and is not suppressed by averaging.

However, the noise image has random variations and approaches its mean value (zero) by taking the average.

Figure 1 shows a noisy image of the NGC 3749 galaxy.

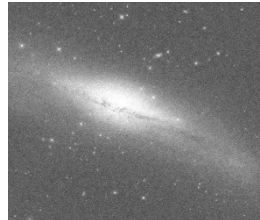


Figure. *Image adapted from Astronomy.com.*

The noise has created bright spots that make it difficult to recognize the individual stars. If we capture this image many times and apply the averaging technique, the bright regions of noise will fade away and we'll be able to more easily recognize the individual stars. The result of averaging 500 images is shown in Figure 2.



Figure 2. *Image adapted from Astronomy.com.*

It can be shown that averaging M noisy images reduces the noise variance by a factor of M . In other words, it increases the signal-to-noise ratio (SNR) by a factor of M .

Image Subtraction

Subtraction of $I_2[x, y]$ from $I_1[x, y]$ is represented by:

$$D[x, y] = I_1[x, y] - I_2[x, y]$$

Note that the subtraction operation is performed on a point by point basis as well. This operation has several interesting applications, including correcting uneven illuminance and comparing images.

Image Subtraction for Correcting Uneven Illuminance

First, let's look at the application of the subtraction operation in mitigating the effect of uneven illuminance. As an example, consider the light microscope image of collenchyma cells shown in Figure 3.

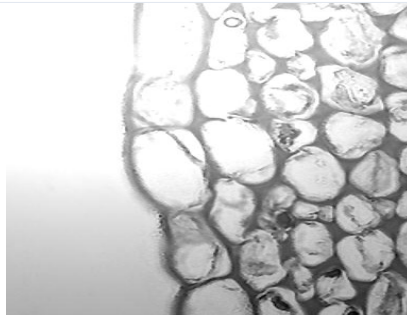


Figure 3. Image adapted from Siyavula.

As you can see, the top left corner of the image is much brighter than the rest of it.

To correct this undesired effect, we can create a reference image that has the same illumination variation. This can be achieved by capturing the image of a uniform scene (e.g. a white sheet of paper). Such a reference image for the example of Figure 3 is shown in Figure 4.

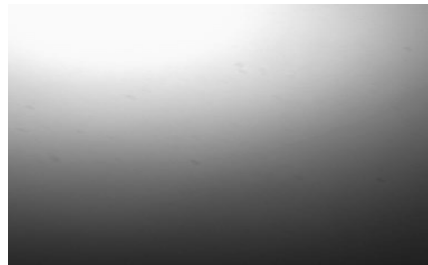


Figure4. A reference image with a corresponding level of illumination variation as seen in Figure 3

If we subtract this reference from the image in Figure 3, a relatively larger value will be subtracted from those regions that are strongly illuminated. This means that the pixel values of the brighter regions, such as the top left corner, will decrease much more than the pixel values in the weakly illuminated areas.

The subtraction operation can lead to negative pixel values in the difference image. If the negative values are not supported by our image format, we have to add a sufficiently large constant to the minuend image (the image in Figure 3) to avoid negative values in the difference image. However, adding this constant to the input image can lead to overflow because, in practice, we are using a limited number of bits to represent each pixel value.

Because of this, before adding the constant, we might need to change our data type to a larger type so as to avoid overflow. For example, while we usually use eight-bit data types to represent the pixel values of a grayscale image, we might need to use a 16-bit data type to successfully perform the calculations.

In this example, we add a constant of 109 to the minuend image and perform the subtraction (Figure 4 is subtracted from Figure 3 plus 109). The result is shown in Figure 5.

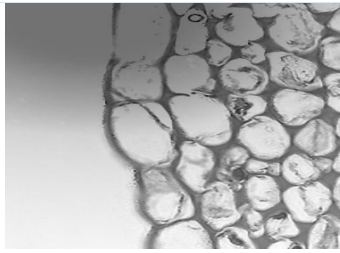


Figure 5

Image Smoothing

Smoothing is used to reduce noise or to produce a less pixelated image. Most smoothing methods are based on low-pass filters, but you can also smooth an image using an average or median value of a group of pixels (a *kernel*) that moves through the image.

The aim of image smoothing is to diminish the effects of camera noise, spurious pixel values, missing pixel values etc. There are many different techniques for image smoothing; we will consider neighbourhood averaging and edge-preserving smoothing.

Image filtering is a most important part of the smoothing process. Image smoothing filters, which include the Gaussian, Maximum, Mean, Median, Minimum, Non-Local Means, Percentile, and Rank filters, can be applied to reduce the amount of noise in an image.

Neighbourhood Averaging

Each point in the smoothed image, $\hat{F}(x, y)$ is obtained from the average pixel value in a neighbourhood of (x, y) in the input image.

For example, if we use a 3×3 neighbourhood around each pixel we would use the mask

```

1 1 1
/ / /
9 9 9
1 1 1

```

```

/ / /
9 9 9
1 1 1
/ / /
9 9 9

```

Each pixel value is multiplied by $\frac{1}{9}$, summed, and then the result placed in the output image. This mask is successively moved across the image until every pixel has been covered. That is, the image is *convolved* with this smoothing mask (also known as a spatial filter or kernel).

However, one usually expects the value of a pixel to be more closely related to the values of pixels close to it than to those further away. This is because most points in an image are spatially coherent with their neighbours; indeed it is generally only at edge or feature points where this hypothesis is not valid. Accordingly it is usual to weight the pixels near the centre of the mask more strongly than those at the edge.

Some common weighting functions include the rectangular weighting function above (which just takes the average over the window), a triangular weighting function, or a Gaussian.

In practice one doesn't notice much difference between different weighting functions, although Gaussian smoothing is the most commonly used. Gaussian smoothing has the attribute that the frequency components of the image are modified in a smooth manner.

Smoothing reduces or attenuates the higher frequencies in the image. Mask shapes other than the Gaussian can do odd things to the frequency spectrum, but as far as the appearance of the image is concerned we usually don't notice much.

Edge preserving smoothing

Neighbourhood averaging or Gaussian smoothing will tend to blur edges because the high frequencies in the image are attenuated. An alternative approach is to use *median filtering*. Here we set the grey level to be the median of the pixel values in the neighbourhood of that pixel.

The median m of a set of values is such that half the values in the set are less than m and half are greater.

For example, suppose the pixel values in a 3×3 neighbourhood are (10, 20, 20, 15, 20, 20, 20, 25, 100). If we sort the values we get (10, 15, 20, 20, |20|, 20, 20, 25, 100) and the median here is 20.

The outcome of median filtering is that pixels with outlying values are forced to become more like their neighbours, but at the same time edges are preserved. Of course, median filters are non-linear.

Median filtering is in fact a morphological operation. When we erode an image, pixel values are replaced with the smallest value in the neighbourhood. Dilating an image corresponds to replacing pixel values

with the largest value in the neighbourhood. Median filtering replaces pixels with the median value in the neighbourhood. It is the rank of the value of the pixel used in the neighbourhood that determines the type of morphological operation.

Figure 3: Image of Genevieve; with salt and pepper noise; the result of averaging; and the result of median filtering.

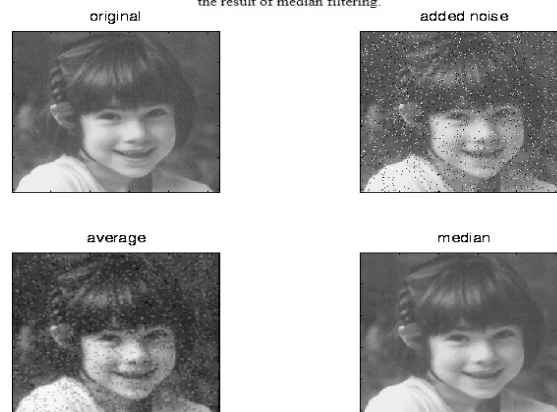
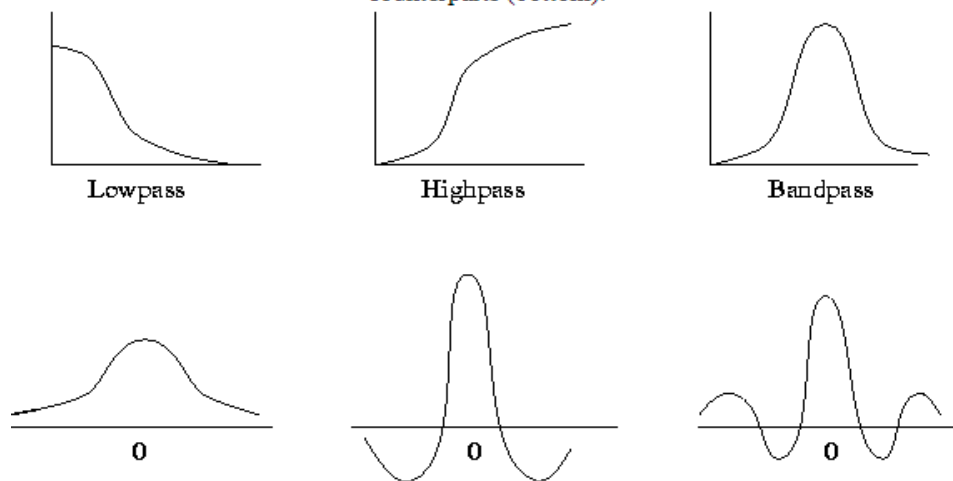


Image sharpening

The main aim in image sharpening is to highlight fine detail in the image, or to enhance detail that has been blurred (perhaps due to noise or other effects, such as motion). With image sharpening, we want to enhance the high-frequency components; this implies a spatial filter shape that has a high positive component at the centre (see figure 4 below).

Figure 4: Frequency domain filters (top) and their corresponding spatial domain counterparts (bottom).



A simple spatial filter that achieves image sharpening is given by

$$\begin{bmatrix} -1/9 & -1/9 & -1/9 \\ -1/9 & 8/9 & -1/9 \\ -1/9 & -1/9 & -1/9 \end{bmatrix}$$

Since the sum of all the weights is zero, the resulting signal will have a zero DC value (that is, the average signal value, or the coefficient of the zero frequency term in the Fourier expansion). For display purposes, we might want to add an offset to keep the result in the $0 \dots 255$ range.

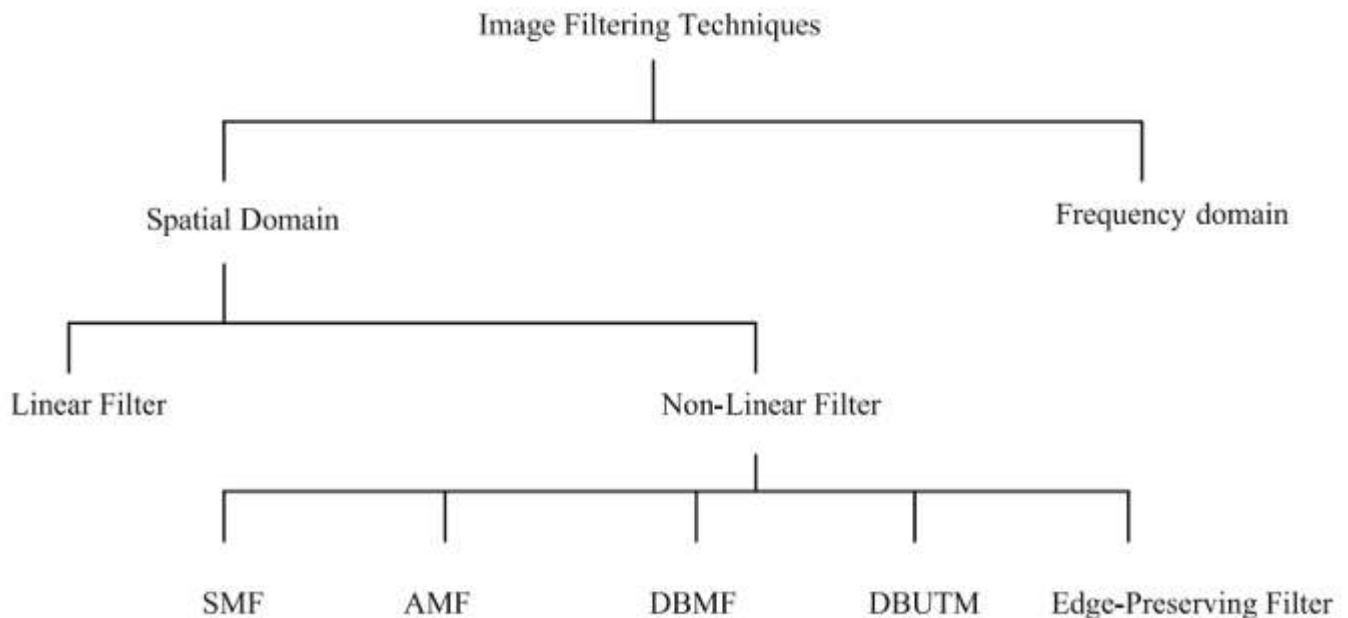
FREQUENCY DOMAIN METHODS

Image enhancement in the frequency domain is straightforward. We simply compute the Fourier transform of the image to be enhanced, multiply the result by a filter (rather than convolve in the spatial domain), and take the inverse transform to produce the enhanced image.

The idea of blurring an image by reducing its high frequency components, or sharpening an image by increasing the magnitude of its high frequency components is intuitively easy to understand. However, computationally, it is often more efficient to implement these operations as convolutions by small spatial filters in the spatial domain. Understanding frequency domain concepts is important, and leads to enhancement techniques that might not have been thought of by restricting attention to the spatial domain.

Filtering techniques

Filtering techniques are used to enhance and modify digital images. Also, image filters are used for blurring and noise reduction, sharpening and edge detection. Image filters are mainly used to suppress high (smoothing techniques) and low frequencies (image enhancement, edge detection). Classification of image filters is as follows.



source: <https://www.researchgate.net/publication/328619526>

Simple Adaptive Median filter (AMF) , Decision Based Median Filter (DBMF) , Decision Based Untrimmed Median Filter (DBUTM)

According to this classification, image filters can be divide in to two main categories. Spatial filtering is the traditional method of image filtering. it is use directly on the image pixels. Frequency domain filters are use to remove high and low frequencies and smoothing.

Non linear filters are use to detect edges. Those filtering techniques are more effective than linear filters. In linear filtering, image details and edges are tend to blur. Gaussian filter, Laplacian filter and Neighborhood Average (Mean) filter can be identify as examples for linear filters. Median filters are non linear filters. The next part of this article is the discussion about different linear and non linear filters.

MEDIAN FILTER...

Median filter is a non-linear filter. It replaces each pixel values by the median values of it's neighbor pixels. This is the efficient way for remove salt-and-pepper noise. The calculation of the median value is given below.

30	35	40	42	42
35	42	37	37	40
38	39	40	41	42
40	41	42	43	43
42	43	45	44	46

		41		

37, 37, 39, 40, **41**, 42, 42, 43



LAPLACIAN FILTER...

Laplace smoothing technique is mainly use to detect image edges. It highlights gray level discontinuities. It is based on second spatial derivation of an image. To define Laplacian operator, below equation has been used.

$$\text{Laplace}(f) = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

Laplace edge detector use only one kernel. To detect the edges of an image, this kernel detects 2nd order derivatives of image's intensity levels by using only single pass. We can use "kernel 2" for detects edges

with diagonals. It will give better approximation. Also, Laplace method gives faster calculations than others.

0	-1	0
-1	4	-1
0	-1	0

Kernel 1

-1	-1	-1
-1	8	-1
-1	-1	-1

Kernel 2



Kernel 1



Kernel 2

GAUSSIAN FILTER...

This filter is a 2-D convolutional operator. It use to blur images. Also, it removes details and noises.

Gaussian filter is similar to mean filter. But main difference is, Gaussian filter use a kernel. That kernel has a shape of gaussian hump. Gaussian kernel weights pixels at its center much more strongly than its boundaries. There are different gaussian kernels. Based on the kernel size, output image will be different.

1/16

1	2	1
2	4	2
1	2	1

3x3 kernel

1/173

1	4	7	4	1
4	16	26	16	4
7	26	41	26	7
4	16	26	16	4
1	4	7	4	1

5x5 kernel



NEIGHBOURHOOD AVERAGE FILTER...

This filter is also called as mean filter. In average filtering, pixel values will be replaced by average values of neighbour pixels. The calculation of average value is as follows.

23	25	30	35	30
25	30	35	37	40
45	40	37	43	45
38	40	43	42	46
35	40	42	45	47

		39		

$$\text{Value} = (30 + 35 + 37 + 40 + 37 + 43 + 40 + 43 + 42) / 9 = 38.55 = 39$$

Spatial Filtering technique

Spatial Filtering technique is used directly on pixels of an image. Mask is usually considered to be added in size so that it has specific center pixel. This mask is moved on the image such that the center of the mask traverses all image pixels.

Classification on the basis of linearity:

There are two types:

1. Linear Spatial Filter
2. Non-linear Spatial Filter

General Classification:

Smoothing Spatial Filter: Smoothing filter is used for blurring and noise reduction in the image.

Blurring is pre-processing steps for removal of small details and Noise Reduction is accomplished by blurring.

Types of Smoothing Spatial Filter:

1. Linear Filter (Mean Filter)
2. Order Statistics (Non-linear) filter

These are explained as following below.

1. **Mean Filter:**

Linear spatial filter is simply the average of the pixels contained in the neighborhood of the filter mask. The idea is replacing the value of every pixel in an image by the average of the grey levels in the neighborhood define by the filter mask.

Types of Mean filter:

- **(i) Averaging filter:** It is used in reduction of the detail in image. All coefficients are equal.
- **(ii) Weighted averaging filter:** In this, pixels are multiplied by different coefficients. Center pixel is multiplied by a higher value than average filter.

2. **Order Statistics Filter:**

It is based on the ordering the pixels contained in the image area encompassed by the filter. It replaces the value of the center pixel with the value determined by the ranking result. Edges are better preserved in this filtering.

Types of Order statistics filter:

- **(i) Minimum filter:** 0th percentile filter is the minimum filter. The value of the center is replaced by the smallest value in the window.
- **(ii) Maximum filter:** 100th percentile filter is the maximum filter. The value of the center is replaced by the largest value in the window.
- **(iii) Median filter:** Each pixel in the image is considered. First neighboring pixels are sorted and original values of the pixel is replaced by the median of the list.

Sharpening Spatial Filter: It is also known as derivative filter. The purpose of the sharpening spatial filter is just the opposite of the smoothing spatial filter. Its main focus in on the removal of blurring and highlight the edges. It is based on the first and second order derivative.

First order derivative:

- Must be zero in flat segments.
- Must be non zero at the onset of a grey level step.
- Must be non zero along ramps.

First order derivative in 1-D is given by:

$$f' = f(x+1) - f(x)$$

Second order derivative:

- Must be zero in flat areas.
- Must be zero at the onset and end of a ramp.

- Must be zero along ramps.

Second order derivative in 1-D is given by:

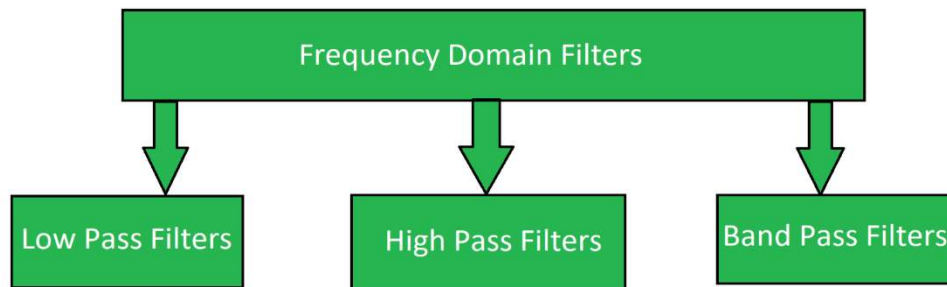
$$f'' = f(x+1) + f(x-1) - 2f(x)$$

Frequency Domain Filters:

Frequency Domain Filters are used for smoothing and sharpening of image by removal of high or low frequency components. Sometimes it is possible of removal of very high and very low frequency.

Frequency domain filters are different from spatial domain filters as it basically focuses on the frequency of the images. It is basically done for two basic operation i.e., Smoothing and Sharpening.

These are of 3 types:



Classification of Frequency Domain Filters

1. Low pass filter:

Low pass filter removes the high frequency components that means it keeps low frequency components. It is used for smoothing the image. It is used to smoothen the image by attenuating high frequency components and preserving low frequency components.

Mechanism of low pass filtering in frequency domain is given by:

$$G(u, v) = H(u, v) \cdot F(u, v)$$

where $F(u, v)$ is the Fourier Transform of original image

and $H(u, v)$ is the Fourier Transform of filtering mask

Low pass filtering involves the elimination of the high frequency components in the image. It results in blurring of the image (and thus a reduction in sharp transitions associated with noise). An ideal low pass filter would retain all the low frequency components, and eliminate all the high frequency components. However, ideal filters suffer from two problems: *blurring* and *ringing*. These problems are caused by the shape of the associated spatial domain filter, which has a large number of undulations. Smoother transitions in the frequency domain filter, such as the Butterworth filter, achieve much better results.

Figure 5: Transfer function for an ideal low pass filter.



2. High pass filter:

High pass filter removes the low frequency components that means it keeps high frequency components. It is used for sharpening the image. It is used to sharpen the image by attenuating low frequency components and preserving high frequency components.

Mechanism of high pass filtering in frequency domain is given by:

$$H(u, v) = 1 - H'(u, v)$$

where $H(u, v)$ is the Fourier Transform of high pass filtering

and $H'(u, v)$ is the Fourier Transform of low pass filtering

3. Band pass filter:

Band pass filter removes the very low frequency and very high frequency components that means it keeps the moderate range band of frequencies. Band pass filtering is used to enhance edges while reducing the noise at the same time.

Filtering

Image filtering can be grouped in two depending on the effects:

1. Low pass filters (Smoothing)

Low pass filtering (aka smoothing), is employed to remove high spatial frequency noise from a digital image. The low-pass filters usually employ moving window operator which affects one pixel of the image at a time, changing its value by some function of a local region (window) of pixels. The operator moves over the image to affect all the pixels in the image.

2. High pass filters (Edge Detection, Sharpening)

A high-pass filter can be used to make an image appear sharper. These filters emphasize fine details in the image - the opposite of the low-pass filter. High-pass filtering works in the same way as low-pass filtering; it just uses a different convolution kernel.

Homomorphic filtering

Homomorphic filtering is a generalized technique for signal and image processing, involving a nonlinear mapping to a different domain in which linear filter techniques are applied, followed by mapping back to

the original domain. This concept was developed in the 1960s by [Thomas Stockham](#), [Alan V. Oppenheim](#), and [Ronald W. Schafer](#) at [MIT](#) and independently by Bogert, Healy, and Tukey in their study of time series.

Homomorphic filtering is sometimes used for [image enhancement](#). It simultaneously normalizes the brightness across an image and increases contrast. Here homomorphic filtering is used to remove [multiplicative noise](#). Illumination and reflectance are not separable, but their approximate locations in the frequency domain may be located. Since illumination and reflectance combine multiplicatively, the components are made additive by taking the [logarithm](#) of the image intensity, so that these multiplicative components of the image can be separated linearly in the frequency domain. Illumination variations can be thought of as a multiplicative noise, and can be reduced by filtering in the log domain.

To make the illumination of an image more even, the high-frequency components are increased and low-frequency components are decreased, because the high-frequency components are assumed to represent mostly the reflectance in the scene (the amount of light reflected off the object in the scene), whereas the low-frequency components are assumed to represent mostly the illumination in the scene. That is, [high-pass filtering](#) is used to suppress low frequencies and amplify high frequencies, in the log-intensity domain.^[3]

Operation

Homomorphic filtering can be used for improving the appearance of a grayscale image by simultaneous intensity range compression (illumination) and contrast enhancement (reflection).

$$m(x, y) = i(x, y) \bullet r(x, y)$$

Where,

m = image,

i = illumination,

r = reflectance

We have to transform the equation into frequency domain in order to apply high pass filter. However, it's very difficult to do calculation after applying Fourier transformation to this equation because it's not a product equation anymore. Therefore, we use 'log' to help solving this problem.

$$\ln(m(x, y)) = \ln(i(x, y)) + \ln(r(x, y))$$

Then, applying Fourier transformation

$$F(\ln(m(x, y))) = F(\ln(i(x, y))) + F(\ln(r(x, y)))$$

$$\text{Or } M(u, v) = I(u, v) + R(u, v)$$

Next, applying high-pass filter to the image. To make the illumination of an image more even, the high-frequency components are increased and low-frequency components are decrease.

$$N(u, v) = H(u, v) \bullet M(u, v)$$

Where

H = any high-pass filter

N = filtered image in frequency domain

Afterward, returning frequency domain back to the spatial domain by using inverse Fourier transform.

$$n(x, y) = \text{inv}F(N(u, v))$$

Finally, using exponential function to eliminate the log we used at the beginning to get the enhanced image

$$\text{newImage}(x, y) = \exp(n(x, y))$$

Scale-Space Analysis-

Scale-space theory is a framework for [multi-scale signal representation](#) developed by the [computer vision](#), [image processing](#) and [signal processing](#) communities with complementary motivations from [physics](#) and [biological vision](#). It is a formal theory for handling image structures at different [scales](#), by representing an image as a one-parameter family of smoothed images, the **scale-space representation**, parametrized by the size of the [smoothing kernel](#) used for suppressing fine-scale structures. The parameter t in this family is referred to as the *scale parameter*, with the interpretation that image structures of spatial size smaller than about have largely been smoothed away in the scale-space level at scale t .

The main type of scale space is the *linear (Gaussian) scale space*, which has wide applicability as well as the attractive property of being possible to derive from a small set of *scale-space axioms*. The corresponding scale-space framework encompasses a theory for Gaussian derivative operators, which can be used as a basis for expressing a large class of visual operations for computerized systems that process visual information. This framework also allows visual operations to be made *scale invariant*, which is necessary for dealing with the size variations that may occur in image data, because

real-world objects may be of different sizes and in addition the distance between the object and the camera may be unknown and may vary depending on the circumstances.

The notion of scale space applies to signals of arbitrary numbers of variables. The most common case in the literature applies to two-dimensional images, which is what is presented here. For a given image $f(x, y)$, its linear (Gaussian) scale-space representation is a family of derived signals $L(x, y; t)$ defined by the convolution of $f(x, y)$ with the two-dimensional Gaussian kernel

$$g(x, y; t) = \frac{1}{2\pi t} e^{-(x^2+y^2)/2t}$$

such that

$$L(\cdot, \cdot; t) = g(\cdot, \cdot; t) * f(\cdot, \cdot),$$

where the semicolon in the argument of L implies that the convolution is performed only over the variables x, y , while the scale parameter t after the semicolon just indicates which scale level is being defined. This definition of L works for a continuum of scales $t \geq 0$, but typically only a finite discrete set of levels in the scale-space representation would be actually considered.

The scale parameter $t = \sigma^2$ is the variance of the Gaussian filter and as a limit for $t = 0$ the filter g becomes an impulse function such that $L(x, y; 0) = f(x, y)$, that is, the scale-space representation at scale level $t = 0$ is the image f itself. As t increases, L is the result of smoothing f with a larger and larger filter, thereby removing more and more of the details that the image contains. Since the standard deviation of the filter is $\sigma = \sqrt{t}$, details that are significantly smaller than this value are to a large extent removed from the image at scale parameter t , see the following figure and^[11] for graphical illustrations.

Gabor filter

In image processing, a **Gabor filter**, named after [Dennis Gabor](#), is a [linear filter](#) used for [texture](#) analysis, which essentially means that it analyzes whether there is any specific frequency content in the image in specific directions in a localized region around the point or region of analysis. Frequency and orientation representations of Gabor filters are claimed by many contemporary vision scientists to be similar to those of the [human visual system](#).^[1] They have been found to be particularly appropriate for texture representation and discrimination. In the spatial domain, a 2-D Gabor filter is a [Gaussian kernel function](#) modulated by a [sinusoidal plane wave](#) (see [Gabor transform](#)).

Some authors claim that simple cells in the [visual cortex](#) of [mammalian brains](#) can be modeled by Gabor functions. Thus, [image analysis](#) with Gabor filters is thought by some to be similar to perception in the [human visual system](#).

Its [impulse response](#) is defined by a [sinusoidal](#) wave (a [plane wave](#) for 2-D Gabor filters) multiplied by a [Gaussian function](#). Because of the multiplication-convolution property ([Convolution theorem](#)), the [Fourier transform](#) of a Gabor filter's impulse response is the [convolution](#) of the Fourier transform of the harmonic function (sinusoidal function) and the Fourier transform of the Gaussian function. The filter

has a [real](#) and an [imaginary](#) component representing [orthogonal](#) directions. The two components may be formed into a [complex number](#) or used individually.

Complex

$$g(x, y; \lambda, \theta, \psi, \sigma, \gamma) = \exp\left(-\frac{x'^2 + \gamma^2 y'^2}{2\sigma^2}\right) \exp\left(i\left(2\pi\frac{x'}{\lambda} + \psi\right)\right)$$

Real

$$g(x, y; \lambda, \theta, \psi, \sigma, \gamma) = \exp\left(-\frac{x'^2 + \gamma^2 y'^2}{2\sigma^2}\right) \cos\left(2\pi\frac{x'}{\lambda} + \psi\right)$$

Imaginary

$$g(x, y; \lambda, \theta, \psi, \sigma, \gamma) = \exp\left(-\frac{x'^2 + \gamma^2 y'^2}{2\sigma^2}\right) \sin\left(2\pi\frac{x'}{\lambda} + \psi\right)$$

Where:

$$x' = x \cos \theta + y \sin \theta \text{ and } y' = -x \sin \theta + y \cos \theta.$$

In this equation, λ represents the wavelength of the sinusoidal factor, θ represents the orientation of the normal to the parallel stripes of a [Gabor function](#), ψ is the phase offset, σ is the sigma/standard deviation of the Gaussian envelope and γ is the spatial aspect ratio, and specifies the ellipticity of the support of the Gabor function.